```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\sikka> mongosh
Current Mongosh Log ID: 6abb3bebfb35ee8a0475ced4
Connecting to:
mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appNa
me=mongosh+2.12.0
Using MongoDB:          8.3.11
Using Mongosh:          2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


------
   The server generated these startup warnings when booting
   2026-09-28T14:37:58.666+05:30: Access control is not enabled for the database. Read and
write access to data and configuration is unrestricted
------

test> show dbs
admin   40.00 KiB
config  48.00 KiB
local   40.00 KiB
test> use studentDB
switched to db studentDB
studentDB> db.createCollection("students")
{ ok: 1 }
studentDB> db.students.insertOne({
|   name: "Aarav Mehta",
|   age: 21,
|   course: "MERN Stack",
|   status: "enrolled"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6abb3c59fb35ee8a0475ced5')
}
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abb3c59fb35ee8a0475ced5'),
    name: 'Aarav Mehta',
    age: 21,
    course: 'MERN Stack',
```

```
    status: 'enrolled'
  }
]
studentDB> db.students.insertMany([
|   { name: "Diya Patel",  age: 20, course: "Data Science",   status: "enrolled" },
|   { name: "Rohan Gupta", age: 23, course: "MERN Stack",     status: "enrolled" },
|   { name: "Sneha Iyer",  age: 22, course: "Python Backend", status: "completed" },
|   { name: "Karan Singh", age: 19, course: "MERN Stack",     status: "enrolled" },
|   { name: "Meera Nair",  age: 24, course: "Data Science",   status: "completed" },
|   { name: "Aditya Rao",  age: 21, course: "React Frontend", status: "enrolled" },
|   { name: "Fatima Khan", age: 22, course: "Python Backend", status: "pending" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abb3c8cfb35ee8a0475ced6'),
    '1': ObjectId('6abb3c8cfb35ee8a0475ced7'),
    '2': ObjectId('6abb3c8cfb35ee8a0475ced8'),
    '3': ObjectId('6abb3c8cfb35ee8a0475ced9'),
    '4': ObjectId('6abb3c8cfb35ee8a0475ceda'),
    '5': ObjectId('6abb3c8cfb35ee8a0475cedb'),
    '6': ObjectId('6abb3c8cfb35ee8a0475cedc')
  }
}
studentDB> db.students.insertOne({
|   name: "Zoya Sheikh",
|   age: 20,
|   course: "React Frontend",
|   status: "enrolled",
|   email: "zoya@mail.com"
| })
{
  acknowledged: true,
  insertedId: ObjectId('6abb3ca8fb35ee8a0475cedd')
}
studentDB> db.students.countDocuments()
9
studentDB> db.students.find()
[
  {
    _id: ObjectId('6abb3c59fb35ee8a0475ced5'),
    name: 'Aarav Mehta',
    age: 21,
    course: 'MERN Stack',
```

```
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced6'),
  name: 'Diya Patel',
  age: 20,
  course: 'Data Science',
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced7'),
  name: 'Rohan Gupta',
  age: 23,
  course: 'MERN Stack',
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced8'),
  name: 'Sneha Iyer',
  age: 22,
  course: 'Python Backend',
  status: 'completed'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced9'),
  name: 'Karan Singh',
  age: 19,
  course: 'MERN Stack',
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ceda'),
  name: 'Meera Nair',
  age: 24,
  course: 'Data Science',
  status: 'completed'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475cedb'),
  name: 'Aditya Rao',
  age: 21,
  course: 'React Frontend',
  status: 'enrolled'
},
```

```
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475cedc'),
    name: 'Fatima Khan',
    age: 22,
    course: 'Python Backend',
    status: 'pending'
  },
  {
    _id: ObjectId('6abb3ca8fb35ee8a0475cedd'),
    name: 'Zoya Sheikh',
    age: 20,
    course: 'React Frontend',
    status: 'enrolled',
    email: 'zoya@mail.com'
  }
]
studentDB> db.students.findOne({ name: "Sneha Iyer" })
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced8'),
  name: 'Sneha Iyer',
  age: 22,
  course: 'Python Backend',
  status: 'completed'
}
studentDB> db.students.find({ course: "MERN Stack" })
[
  {
    _id: ObjectId('6abb3c59fb35ee8a0475ced5'),
    name: 'Aarav Mehta',
    age: 21,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced7'),
    name: 'Rohan Gupta',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced9'),
    name: 'Karan Singh',
    age: 19,
```

```
    course: 'MERN Stack',
    status: 'enrolled'
  }
]
studentDB> db.students.find({ course: "Python Backend", status: "completed" })
[
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced8'),
    name: 'Sneha Iyer',
    age: 22,
    course: 'Python Backend',
    status: 'completed'
  }
]
studentDB> db.students.find(
|   { course: "MERN Stack" },
|   { name: 1, status: 1, _id: 0 }
| )
[
  { name: 'Aarav Mehta', status: 'enrolled' },
  { name: 'Rohan Gupta', status: 'enrolled' },
  { name: 'Karan Singh', status: 'enrolled' }
]
studentDB> db.students.find().sort({ age: -1 })
[
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ceda'),
    name: 'Meera Nair',
    age: 24,
    course: 'Data Science',
    status: 'completed'
  },
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced7'),
    name: 'Rohan Gupta',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced8'),
    name: 'Sneha Iyer',
    age: 22,
    course: 'Python Backend',
```

```
  status: 'completed'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475cedc'),
  name: 'Fatima Khan',
  age: 22,
  course: 'Python Backend',
  status: 'pending'
},
{
  _id: ObjectId('6abb3c59fb35ee8a0475ced5'),
  name: 'Aarav Mehta',
  age: 21,
  course: 'MERN Stack',
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475cedb'),
  name: 'Aditya Rao',
  age: 21,
  course: 'React Frontend',
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced6'),
  name: 'Diya Patel',
  age: 20,
  course: 'Data Science',
  status: 'enrolled'
},
{
  _id: ObjectId('6abb3ca8fb35ee8a0475cedd'),
  name: 'Zoya Sheikh',
  age: 20,
  course: 'React Frontend',
  status: 'enrolled',
  email: 'zoya@mail.com'
},
{
  _id: ObjectId('6abb3c8cfb35ee8a0475ced9'),
  name: 'Karan Singh',
  age: 19,
  course: 'MERN Stack',
  status: 'enrolled'
```

```
  }
]
studentDB> db.students.find({ status: "enrolled" }).sort({ age: 1 }).limit(3)
[
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced9'),
    name: 'Karan Singh',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3ca8fb35ee8a0475cedd'),
    name: 'Zoya Sheikh',
    age: 20,
    course: 'React Frontend',
    status: 'enrolled',
    email: 'zoya@mail.com'
  },
  {
    _id: ObjectId('6abb3c8cfb35ee8a0475ced6'),
    name: 'Diya Patel',
    age: 20,
    course: 'Data Science',
    status: 'enrolled'
  }
]
studentDB> db.students.countDocuments({ status: "enrolled" })
6
studentDB> db.students.updateOne({ name: "Karan Singh" }, { status: "completed" })
MongoInvalidArgumentError: Update document requires atomic operators
studentDB> db.students.updateOne(
|   { name: "Karan Singh" },
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany(
|   { course: "Data Science" },
```

```
|   { $set: { status: "completed" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany(
|   {},
|   { $set: { batch: "2024-B" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 9,
  modifiedCount: 9,
  upsertedCount: 0
}
studentDB> db.students.updateOne({ name: "Aarav Mehta" }, { $inc: { age: 1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateOne({ name: "Aarav Mehta" }, { $push: { skills: "JavaScript" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.deleteOne({ name: "Aditya Rao" })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({ status: "pending" })
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 7 }
studentDB> db.students.insertMany([
```

```
|   { name: "Aarav Mehta",  age: 21, course: "MERN Stack",     status: "completed", email:
"aarav@mail.com" },
|   { name: "Diya Patel",   age: 20, course: "Data Science",   status: "enrolled",  email:
"diya@mail.com" },
|   { name: "Rohan Gupta",  age: 23, course: "MERN Stack",     status: "enrolled" },
|   { name: "Sneha Iyer",   age: 22, course: "Python Backend", status: "completed", email:
"sneha@mail.com" },
|   { name: "Karan Singh",  age: 19, course: "MERN Stack",     status: "enrolled",  email:
"karan@mail.com" },
|   { name: "Meera Nair",   age: 24, course: "Data Science",   status: "enrolled" },
|   { name: "Aditya Rao",   age: 21, course: "React Frontend", status: "completed" },
|   { name: "Fatima Khan",  age: 22, course: "Python Backend", status: "enrolled",  email:
"fatima@mail.com" },
|   { name: "Vivaan Joshi", age: 25, course: "MERN Stack",     status: "pending" },
|   { name: "Ananya Das",   age: 20, course: "Data Science",   status: "pending",   email:
"ananya@mail.com" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abb3e84fb35ee8a0475cede'),
    '1': ObjectId('6abb3e84fb35ee8a0475cedf'),
    '2': ObjectId('6abb3e84fb35ee8a0475cee0'),
    '3': ObjectId('6abb3e84fb35ee8a0475cee1'),
    '4': ObjectId('6abb3e84fb35ee8a0475cee2'),
    '5': ObjectId('6abb3e84fb35ee8a0475cee3'),
    '6': ObjectId('6abb3e84fb35ee8a0475cee4'),
    '7': ObjectId('6abb3e84fb35ee8a0475cee5'),
    '8': ObjectId('6abb3e84fb35ee8a0475cee6'),
    '9': ObjectId('6abb3e84fb35ee8a0475cee7')
  }
}
studentDB> db.students.find({ age: { $gt: 21 } })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee0'),
    name: 'Rohan Gupta',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee1'),
    name: 'Sneha Iyer',
```

```
    age: 22,
    course: 'Python Backend',
    status: 'completed',
    email: 'sneha@mail.com'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee3'),
    name: 'Meera Nair',
    age: 24,
    course: 'Data Science',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee5'),
    name: 'Fatima Khan',
    age: 22,
    course: 'Python Backend',
    status: 'enrolled',
    email: 'fatima@mail.com'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee6'),
    name: 'Vivaan Joshi',
    age: 25,
    course: 'MERN Stack',
    status: 'pending'
  }
]
studentDB> db.students.find({ age: { $lt: 21 } })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cedf'),
    name: 'Diya Patel',
    age: 20,
    course: 'Data Science',
    status: 'enrolled',
    email: 'diya@mail.com'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee2'),
    name: 'Karan Singh',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled',
```

```
    email: 'karan@mail.com'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee7'),
    name: 'Ananya Das',
    age: 20,
    course: 'Data Science',
    status: 'pending',
    email: 'ananya@mail.com'
  }
]
studentDB> db.students.find({ course: { $in: ["MERN Stack", "Data Science"] } })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cede'),
    name: 'Aarav Mehta',
    age: 21,
    course: 'MERN Stack',
    status: 'completed',
    email: 'aarav@mail.com'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cedf'),
    name: 'Diya Patel',
    age: 20,
    course: 'Data Science',
    status: 'enrolled',
    email: 'diya@mail.com'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee0'),
    name: 'Rohan Gupta',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee2'),
    name: 'Karan Singh',
    age: 19,
    course: 'MERN Stack',
    status: 'enrolled',
    email: 'karan@mail.com'
  },
```

```
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee3'),
    name: 'Meera Nair',
    age: 24,
    course: 'Data Science',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee6'),
    name: 'Vivaan Joshi',
    age: 25,
    course: 'MERN Stack',
    status: 'pending'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee7'),
    name: 'Ananya Das',
    age: 20,
    course: 'Data Science',
    status: 'pending',
    email: 'ananya@mail.com'
  }
]
studentDB> db.students.find({
|   $and: [
|     { age: { $gt: 20 } },
|     { status: "enrolled" }
|   ]
| })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee0'),
    name: 'Rohan Gupta',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee3'),
    name: 'Meera Nair',
    age: 24,
    course: 'Data Science',
    status: 'enrolled'
  },
```

```
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee5'),
    name: 'Fatima Khan',
    age: 22,
    course: 'Python Backend',
    status: 'enrolled',
    email: 'fatima@mail.com'
  }
]
studentDB> db.students.find({
|   $or: [
|     { status: "pending" },
|     { age: { $gte: 24 } }
|   ]
| })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee3'),
    name: 'Meera Nair',
    age: 24,
    course: 'Data Science',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee6'),
    name: 'Vivaan Joshi',
    age: 25,
    course: 'MERN Stack',
    status: 'pending'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee7'),
    name: 'Ananya Das',
    age: 20,
    course: 'Data Science',
    status: 'pending',
    email: 'ananya@mail.com'
  }
]
studentDB> db.students.find({ email: { $exists: true } })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cede'),
    name: 'Aarav Mehta',
```

```
  age: 21,
  course: 'MERN Stack',
  status: 'completed',
  email: 'aarav@mail.com'
},
{
  _id: ObjectId('6abb3e84fb35ee8a0475cedf'),
  name: 'Diya Patel',
  age: 20,
  course: 'Data Science',
  status: 'enrolled',
  email: 'diya@mail.com'
},
{
  _id: ObjectId('6abb3e84fb35ee8a0475cee1'),
  name: 'Sneha Iyer',
  age: 22,
  course: 'Python Backend',
  status: 'completed',
  email: 'sneha@mail.com'
},
{
  _id: ObjectId('6abb3e84fb35ee8a0475cee2'),
  name: 'Karan Singh',
  age: 19,
  course: 'MERN Stack',
  status: 'enrolled',
  email: 'karan@mail.com'
},
{
  _id: ObjectId('6abb3e84fb35ee8a0475cee5'),
  name: 'Fatima Khan',
  age: 22,
  course: 'Python Backend',
  status: 'enrolled',
  email: 'fatima@mail.com'
},
{
  _id: ObjectId('6abb3e84fb35ee8a0475cee7'),
  name: 'Ananya Das',
  age: 20,
  course: 'Data Science',
  status: 'pending',
  email: 'ananya@mail.com'
```

```
  }
]
studentDB> db.students.find({ email: { $exists: false } })
[
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee0'),
    name: 'Rohan Gupta',
    age: 23,
    course: 'MERN Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee3'),
    name: 'Meera Nair',
    age: 24,
    course: 'Data Science',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee4'),
    name: 'Aditya Rao',
    age: 21,
    course: 'React Frontend',
    status: 'completed'
  },
  {
    _id: ObjectId('6abb3e84fb35ee8a0475cee6'),
    name: 'Vivaan Joshi',
    age: 25,
    course: 'MERN Stack',
    status: 'pending'
  }
]
studentDB> db.students.find(
|   { $and: [
|       { course: "MERN Stack" },
|       { status: "enrolled" },
|       { age: { $lt: 25 } }
|   ]},
|   { name: 1, age: 1, _id: 0 }
| ).sort({ age: 1 })
[ { name: 'Karan Singh', age: 19 }, { name: 'Rohan Gupta', age: 23 } ]
studentDB> use libraryDB
| db.createCollection("books")
```

```
switched to db libraryDB
libraryDB> db.books.insertMany([
|   { title: "The Alchemist",            author: "Paulo Coelho",      genre: "Fiction",     price: 299,
copies: 5, available: true,  publishedYear: 1988 },
|   { title: "Atomic Habits",            author: "James Clear",       genre: "Self-Help",   price: 450,
copies: 3, available: true,  publishedYear: 2018 },
|   { title: "Clean Code",               author: "Robert C. Martin",  genre: "Programming", price: 550,
copies: 4, available: true,  publishedYear: 2008 },
|   { title: "Ikigai",                   author: "Hector Garcia",     genre: "Self-Help",   price: 350, copies:
0, available: false, publishedYear: 2016 },
|   { title: "The Pragmatic Programmer", author: "Andrew Hunt",       genre: "Programming", price:
620, copies: 2, available: true,  publishedYear: 1999 },
|   { title: "Sapiens",                  author: "Yuval Noah Harari", genre: "History",     price: 499,
copies: 6, available: true,  publishedYear: 2011 },
|   { title: "Deep Work",                author: "Cal Newport",       genre: "Self-Help",   price: 399,
copies: 0, available: false, publishedYear: 2016 },
|   { title: "Eloquent JavaScript",      author: "Marijn Haverbeke",  genre: "Programming", price:
420, copies: 3, available: true,  publishedYear: 2011 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6abb3f71fb35ee8a0475cee8'),
    '1': ObjectId('6abb3f71fb35ee8a0475cee9'),
    '2': ObjectId('6abb3f71fb35ee8a0475ceea'),
    '3': ObjectId('6abb3f71fb35ee8a0475ceeb'),
    '4': ObjectId('6abb3f71fb35ee8a0475ceec'),
    '5': ObjectId('6abb3f71fb35ee8a0475ceed'),
    '6': ObjectId('6abb3f71fb35ee8a0475ceee'),
    '7': ObjectId('6abb3f71fb35ee8a0475ceef')
  }
}
libraryDB> db.books.insertOne({ title: "Rich Dad Poor Dad", author: "Robert Kiyosaki", genre:
"Self-Help", price: 399, copies: 4, available: true, ppublishedYear: 1997 })
{
  acknowledged: true,
  insertedId: ObjectId('6abb3f7efb35ee8a0475cef0')
}
libraryDB> db.books.find({ genre: "Programming" })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceea'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
```

```
    genre: 'Programming',
    price: 550,
    copies: 4,
    available: true,
    publishedYear: 2008
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceec'),
    title: 'The Pragmatic Programmer',
    author: 'Andrew Hunt',
    genre: 'Programming',
    price: 620,
    copies: 2,
    available: true,
    publishedYear: 1999
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceef'),
    title: 'Eloquent JavaScript',
    author: 'Marijn Haverbeke',
    genre: 'Programming',
    price: 420,
    copies: 3,
    available: true,
    publishedYear: 2011
  }
]
libraryDB> db.books.find({ genre: "Programming" })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceea'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    genre: 'Programming',
    price: 550,
    copies: 4,
    available: true,
    publishedYear: 2008
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceec'),
    title: 'The Pragmatic Programmer',
    author: 'Andrew Hunt',
    genre: 'Programming',
```

```
    price: 620,
    copies: 2,
    available: true,
    publishedYear: 1999
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceef'),
    title: 'Eloquent JavaScript',
    author: 'Marijn Haverbeke',
    genre: 'Programming',
    price: 420,
    copies: 3,
    available: true,
    publishedYear: 2011
  }
]
libraryDB> db.books.find({ available: true })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee8'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    genre: 'Fiction',
    price: 299,
    copies: 5,
    available: true,
    publishedYear: 1988
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee9'),
    title: 'Atomic Habits',
    author: 'James Clear',
    genre: 'Self-Help',
    price: 450,
    copies: 3,
    available: true,
    publishedYear: 2018
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceea'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    genre: 'Programming',
    price: 550,
```

```
  copies: 4,
  available: true,
  publishedYear: 2008
},
{
  _id: ObjectId('6abb3f71fb35ee8a0475ceec'),
  title: 'The Pragmatic Programmer',
  author: 'Andrew Hunt',
  genre: 'Programming',
  price: 620,
  copies: 2,
  available: true,
  publishedYear: 1999
},
{
  _id: ObjectId('6abb3f71fb35ee8a0475ceed'),
  title: 'Sapiens',
  author: 'Yuval Noah Harari',
  genre: 'History',
  price: 499,
  copies: 6,
  available: true,
  publishedYear: 2011
},
{
  _id: ObjectId('6abb3f71fb35ee8a0475ceef'),
  title: 'Eloquent JavaScript',
  author: 'Marijn Haverbeke',
  genre: 'Programming',
  price: 420,
  copies: 3,
  available: true,
  publishedYear: 2011
},
{
  _id: ObjectId('6abb3f7efb35ee8a0475cef0'),
  title: 'Rich Dad Poor Dad',
  author: 'Robert Kiyosaki',
  genre: 'Self-Help',
  price: 399,
  copies: 4,
  available: true,
  publishedYear: 1997
}
```

```
]
libraryDB> db.books.find({ price: { $gt: 300, $lt: 500 } })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee9'),
    title: 'Atomic Habits',
    author: 'James Clear',
    genre: 'Self-Help',
    price: 450,
    copies: 3,
    available: true,
    publishedYear: 2018
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceeb'),
    title: 'Ikigai',
    author: 'Hector Garcia',
    genre: 'Self-Help',
    price: 350,
    copies: 0,
    available: false,
    publishedYear: 2016
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceed'),
    title: 'Sapiens',
    author: 'Yuval Noah Harari',
    genre: 'History',
    price: 499,
    copies: 6,
    available: true,
    publishedYear: 2011
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceee'),
    title: 'Deep Work',
    author: 'Cal Newport',
    genre: 'Self-Help',
    price: 399,
    copies: 0,
    available: false,
    publishedYear: 2016
  },
  {
```

```
    _id: ObjectId('6abb3f71fb35ee8a0475ceef'),
    title: 'Eloquent JavaScript',
    author: 'Marijn Haverbeke',
    genre: 'Programming',
    price: 420,
    copies: 3,
    available: true,
    publishedYear: 2011
  },
  {
    _id: ObjectId('6abb3f7efb35ee8a0475cef0'),
    title: 'Rich Dad Poor Dad',
    author: 'Robert Kiyosaki',
    genre: 'Self-Help',
    price: 399,
    copies: 4,
    available: true,
    publishedYear: 1997
  }
]
libraryDB> db.books.find({ genre: { $in: ["Programming", "History"] } })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceea'),
    title: 'Clean Code',
    author: 'Robert C. Martin',
    genre: 'Programming',
    price: 550,
    copies: 4,
    available: true,
    publishedYear: 2008
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceec'),
    title: 'The Pragmatic Programmer',
    author: 'Andrew Hunt',
    genre: 'Programming',
    price: 620,
    copies: 2,
    available: true,
    publishedYear: 1999
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceed'),
```

```
    title: 'Sapiens',
    author: 'Yuval Noah Harari',
    genre: 'History',
    price: 499,
    copies: 6,
    available: true,
    publishedYear: 2011
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceef'),
    title: 'Eloquent JavaScript',
    author: 'Marijn Haverbeke',
    genre: 'Programming',
    price: 420,
    copies: 3,
    available: true,
    publishedYear: 2011
  }
]
libraryDB> db.books.find({ $or: [ { genre: "Self-Help" }, { price: { $lt: 350 } } ] })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee8'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    genre: 'Fiction',
    price: 299,
    copies: 5,
    available: true,
    publishedYear: 1988
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee9'),
    title: 'Atomic Habits',
    author: 'James Clear',
    genre: 'Self-Help',
    price: 450,
    copies: 3,
    available: true,
    publishedYear: 2018
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceeb'),
    title: 'Ikigai',
```

```
    author: 'Hector Garcia',
    genre: 'Self-Help',
    price: 350,
    copies: 0,
    available: false,
    publishedYear: 2016
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceee'),
    title: 'Deep Work',
    author: 'Cal Newport',
    genre: 'Self-Help',
    price: 399,
    copies: 0,
    available: false,
    publishedYear: 2016
  },
  {
    _id: ObjectId('6abb3f7efb35ee8a0475cef0'),
    title: 'Rich Dad Poor Dad',
    author: 'Robert Kiyosaki',
    genre: 'Self-Help',
    price: 399,
    copies: 4,
    available: true,
    publishedYear: 1997
  }
]
libraryDB> db.books.find({ publishedYear: { $gt: 2010 }, available: true })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee9'),
    title: 'Atomic Habits',
    author: 'James Clear',
    genre: 'Self-Help',
    price: 450,
    copies: 3,
    available: true,
    publishedYear: 2018
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceed'),
    title: 'Sapiens',
    author: 'Yuval Noah Harari',
```

```
    genre: 'History',
    price: 499,
    copies: 6,
    available: true,
    publishedYear: 2011
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceef'),
    title: 'Eloquent JavaScript',
    author: 'Marijn Haverbeke',
    genre: 'Programming',
    price: 420,
    copies: 3,
    available: true,
    publishedYear: 2011
  }
]
libraryDB> db.books.updateOne({ title: "Clean Code" }, { $inc: { copies: -1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> db.books.updateOne(
|   { title: "Atomic Habits" },
|   { $set: { available: false }, $inc: { copies: -3 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> db.books.updateOne(
|   { title: "Atomic Habits" },
|   { $set: { available: true }, $inc: { copies: 3 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
```

```
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> db.books.updateMany({ genre: "Self-Help" }, { $set: { discount: "10%" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4,
  upsertedCount: 0
}
libraryDB> db.books.find({ discount: { $exists: true } })
[
  {
    _id: ObjectId('6abb3f71fb35ee8a0475cee9'),
    title: 'Atomic Habits',
    author: 'James Clear',
    genre: 'Self-Help',
    price: 450,
    copies: 3,
    available: true,
    publishedYear: 2018,
    discount: '10%'
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceeb'),
    title: 'Ikigai',
    author: 'Hector Garcia',
    genre: 'Self-Help',
    price: 350,
    copies: 0,
    available: false,
    publishedYear: 2016,
    discount: '10%'
  },
  {
    _id: ObjectId('6abb3f71fb35ee8a0475ceee'),
    title: 'Deep Work',
    author: 'Cal Newport',
    genre: 'Self-Help',
    price: 399,
    copies: 0,
    available: false,
    publishedYear: 2016,
```

```
    discount: '10%'
  },
  {
    _id: ObjectId('6abb3f7efb35ee8a0475cef0'),
    title: 'Rich Dad Poor Dad',
    author: 'Robert Kiyosaki',
    genre: 'Self-Help',
    price: 399,
    copies: 4,
    available: true,
    publishedYear: 1997,
    discount: '10%'
  }
]
libraryDB> db.books.updateOne({ title: "Sapiens" }, { $push: { tags: "bestseller" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> db.books.deleteOne({ title: "Deep Work" })
{ acknowledged: true, deletedCount: 1 }
libraryDB>
```